מפרקים את שולה המוקשים

מבוא לרברסינג - מטלת סיכום

שלב א' - מגלה המוקשים

המטרה בשלב הזה הייתה לגרום לכך שכבר בפתיחת המשחק כל המוקשים יהיו מסומנים בדגל. כדי לעשות את זה קודם הייתי צריך להבין איפה לוח המשחק נמצא בזיכרון, איך המשחק שומר את המצב של כל תא, ורק אחר כך למצוא את המקום בקוד שבו המוקשים נוצרים ולשנות אותו.

1. איך גרמתי למוקשים להיות מסומנים אוטומטית

הרצתי את המשחק דרך ה - Local Windows Debugger של IDA והסתכלתי על הזיכרון בזמן שהמשחק רץ. אחרי שמצאתי את האזור של לוח המשחק, שמתי hardware breakpoint על אחד התאים בזיכרון כדי לראות איזה קוד משנה אותו. הסעיף הזה מתחיל מהשלב שבו כבר ידעתי איפה הלוח נמצא ומה המשמעות של הערכים בו, כי ככה מסודרות השאלות במטלה. איך מצאתי את הלוח מוסבר בסעיף 2, ומה המשמעות של כל ערך בסעיף 3, וזה גם הסדר שבו עשיתי את זה בפועל. במהלך הבדיקה הגעתי לכתובת: `0x010036FA` ושם נמצאת ההוראה:

```
or byte ptr [eax], 80h
```

הבתים של ההוראה הם:

```
80 08 80
```

בדקתי את ההוראה גם בזמן ריצה. לפני שהיא התבצעה הערך של התא היה `0x0F` , ואחרי שהיא רצה הערך הפך ל - `0x8F` . מזה הבנתי שההוראה הזאת אחראית להוספת המוקש לתא. היא מדליקה את הביט `0x80` , בלי לשנות את שאר הערך של התא. מהבדיקות שעשיתי על הלוח ראיתי ש - `0x8F` הוא מוקש מכוסה, ו - `0x8E` הוא מוקש שמסומן בדגל.

הלולאה שמפזרת את המוקשים

רציתי לוודא שההוראה הזאת היא באמת מה שחשבתי, אז הסתכלתי על כל הקטע שמסביב לכתובת `0x010036FA` . ראיתי שהיא יושבת בתוך לולאה שמפזרת את המוקשים על הלוח:

```
0x010036C7  push dword ptr [1005334h]          ; רוחב הלוח
0x010036CD  call 1003940h                      ; הגרלת מספר אקראי
```

```
0x010036D2  push dword ptr [1005338h]           ; גובה הלוח
0x010036D8  mov  esi, eax
0x010036DA  inc  esi                            ; עמודה אקראית
0x010036DB  call 1003940h
0x010036E0  inc  eax                            ; שורה אקראית
0x010036E1  mov  ecx, eax
0x010036E3  shl  ecx, 5                         ; 32 * שורה
0x010036E6  test byte ptr [ecx+esi+1005340h], 80h
0x010036EE  jne  10036C7h                       ; כבר יש שם מוקש - להגריל מחדש
0x010036F0  shl  eax, 5
0x010036F3  lea  eax, [eax+esi+1005340h]        ; כתובת התא שהוגרל
0x010036FA  or   byte ptr [eax], 80h            ; הנחת המוקש
0x010036FD  dec  dword ptr [1005330h]           ; מוקש אחד פחות להניח
0x01003703  jne  10036C7h                       ; להמשיך עד שנגמרו המוקשים
```

מהלולאה הבנתי כמה דברים. היא מגרילה שורה ועמודה ומחשבת מהן כתובת עם `shl 5` ו - `lea`, וזה התאים למה שכבר ראיתי בזיכרון - הלוח מתחיל ב - `0x01005340` והמרחק בין שורה לשורה הוא 32 בתים. לפני שהיא מניחה מוקש היא בודקת `test ..., 80h` על התא שהוגרל, ואם הביט כבר דלוק היא מגרילה תא אחר, אז הביט `0x80` הוא באמת המוקש ולא רק ניחוש שלי מהזיכרון. וחוץ מזה היא סופרת כמה מוקשים נשארו להניח ב - `0x01005330`, וזה המשתנה שהשתמשתי בו אחר כך בשלב ב'. לכן שיניתי את ההוראה המקורית:

```
or byte ptr [eax], 80h
```

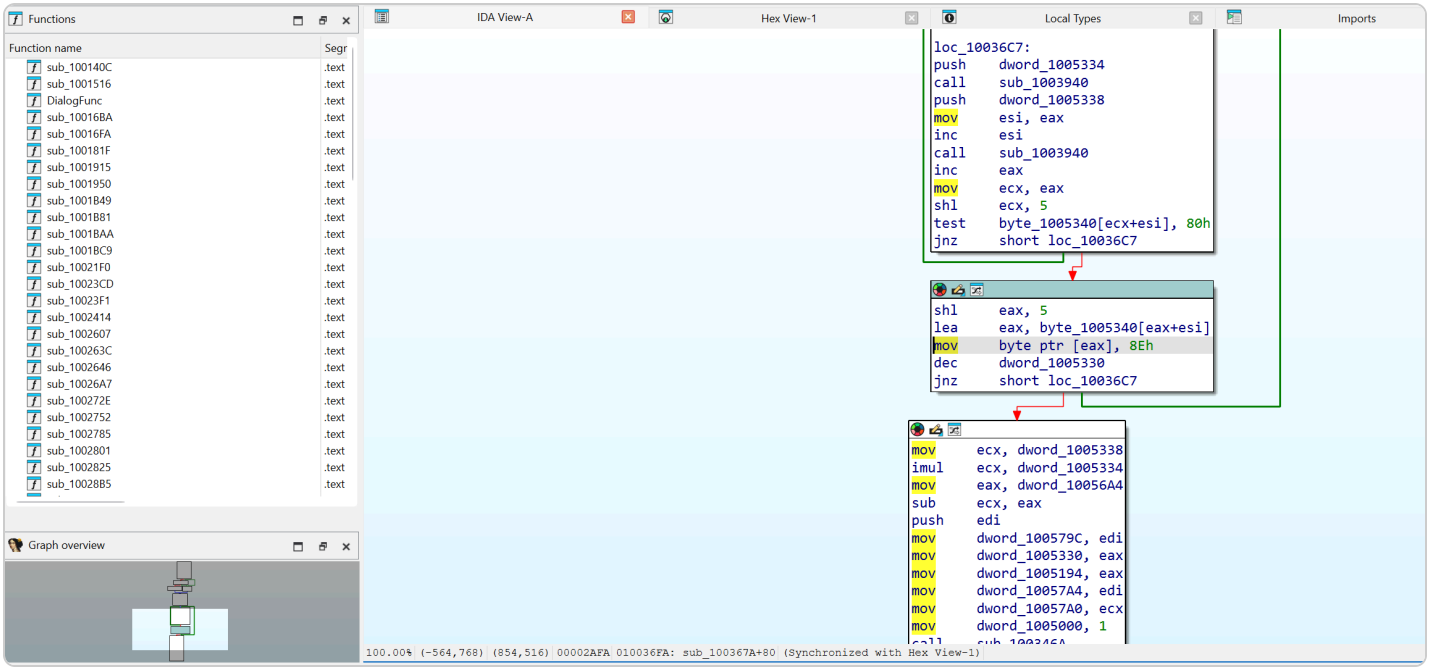
להוראה:

```
mov byte ptr [eax], 8Eh
```

כך שבמקום ליצור מוקש במצב רגיל, המשחק יוצר אותו ישר במצב של מוקש עם דגל. הפאצ' שעשיתי:

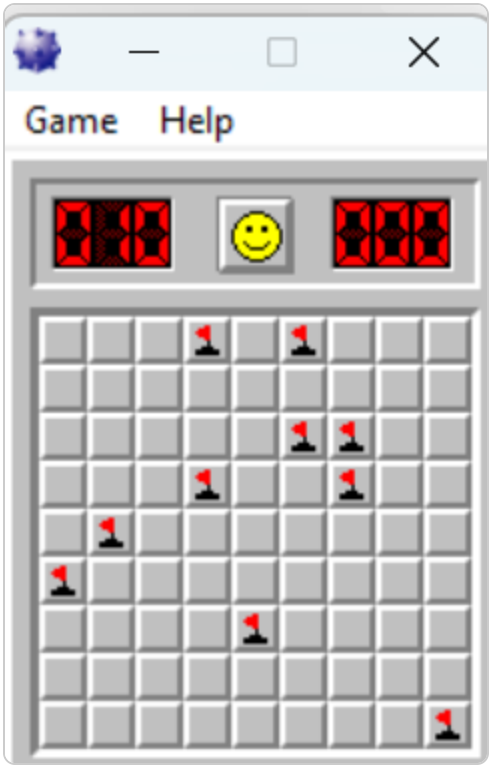
כתובת	לפני	בתים	אחרי	בתים
`0x010036FA`	`or byte ptr [eax], 80h`	`80 08 80`	`mov byte ptr [eax], 8Eh`	`C6 00 8E`

בקובץ עצמו הבתים האלה יושבים ב - offset `0x2AFA`. `0x010036FA` היא כתובת בזיכרון, וההמרה ל - offset בקובץ היא ה - RVA `0x36FA`, פחות ה - RVA של `.text` שהוא `0x1000`, ועוד ה - PointerToRawData שלו שהוא `0x400`. שווה לשים לב שהבתים `80 08 80` מופיעים בקובץ בארבעה מקומות שונים, אז חיפוש של הבתים לבד לא מספיק כדי להגיע להוראה הנכונה.



למה הפאצ' לא שובר משהו

שתי ההוראות הן באורך שלושה בתים, אז לא הייתי צריך להוסיף `nop` ושום דבר אחר בקובץ לא זז. הכתיבה של `0x8E` דורסת את כל הבית ולא רק את הביט של המוקש, אז בדקתי מה יש בתא לפני. הפונקציה שמאתחלת את הלוח, שנקראת ממש לפני הלולאה מהכתובת `0x010036B2`, ממלאת את כל תאי המשחק ב - `0x0F` (ההוראה `mov byte ptr [eax+1005340h], 0Fh` בכתובת `0x01002EDB`). כלומר התא תמיד `0x0F` ברגע הזה, ולכן הכתיבה שלי רק מדליקה את ביט המוקש ומשנה את התצוגה ממכוסה לדגל. גם הבדיקה `test ..., 80h` בתחילת הלולאה ממשיכה לעבוד, כי הביט הזה נשאר דלוק. ככה המשחק לא מניח שני מוקשים באותו תא והלולאה נגמרת אחרי מספר המוקשים הנכון. אחרי ששמרתי את השינוי והרצתי את המשחק מחדש, כל המוקשים הופיעו כבר מסומנים בדגל. בדקתי גם שהמשחק עדיין עובד כרגיל ושאפשר לפתוח תאים רגילים אחרי הפאצ'. בנוסף לחצתי F2 להתחלת משחק חדש, וגם הלוח החדש עלה עם כל המוקשים מסומנים - זה הגיוני, כי הפאצ' יושב בתוך לולאת הפיזור שרצה מחדש בכל משחק.





---

2. איפה נמצא לוח המשחק בזיכרון ואיך מצאתי אותו

איך צמצמתי את שטח החיפוש

הזיכרון של תהליך הוא ענק, ואי אפשר פשוט לפתוח אותו ולחפש מה משתנה - בכל רגע משתנים בו המון דברים שאין להם קשר למשחק. לכן קודם ניסיתי לחשוב איפה הלוח בכלל יכול לשבת. ב - IDA ראיתי שה - ImageBase של WINMINE הוא `0x01000000` . הלוח קיים לאורך כל הריצה, ולכן הנחתי שהוא לא זיכרון שמוקצה תוך כדי ריצה אלא משתנה גלובלי של התוכנית, והתחלתי לחפש אותו ב - `.data` . בטבלת ה - sections ראיתי של - `.data` יש RVA של `0x5000` ו - VirtualSize של `0xB98` . הערך בשדה הזה הוא RVA, כלומר היסט מה - ImageBase של התוכנית בזיכרון ולא כתובת מלאה, ולכן בזמן ריצה ה - section מתחיל ב:

```
0x01000000 + 0x5000 = 0x01005000
```

כלומר כל שטח החיפוש הוא פחות מ - 3 קילובייט. את זה כבר אפשר לעבור בעיניים ב - Hex View, וזה כבר לא חיפוש בכל הזיכרון.

המניפולציה שעשיתי בתוך האזור הזה

פתחתי את האזור ב - Hex View של IDA בזמן שהמשחק רץ, ואז נגעתי בתא אחד בלבד: שמתי עליו דגל, לחצתי שוב והעברתי אותו לסימן שאלה, ולחיצה נוספת החזירה אותו למצב רגיל. בכל לחיצה השתנה בית אחד בדיוק באזור שאני מסתכל עליו, והוא עבר:

```
0x0F -> 0x0E -> 0x0D -> 0x0F
```

אחר כך עשיתי את אותו דבר על תא אחר, והפעם השתנה בית אחר, קרוב לראשון. ככה ידעתי שאני מסתכל על הלוח עצמו ולא על משתנה אחר שמשתנה במקרה באותו זמן. הבית הראשון של האזור הזה נמצא בכתובת: `0x01005340` כל תא בלוח תופס בית אחד בזיכרון. כדי לוודא שזה באמת הלוח שמתי hardware breakpoint מסוג Write על אחד התאים שמצאתי. כשהמשחק שינה את התא, IDA עצרה בקוד שכותב אליו. אחת העצירות הייתה באזור הכתובת: `0x01002ECB` בקוד שמחשב את הכתובת של התא מופיעה גם ההוראה:

```
shl eax, 5
```

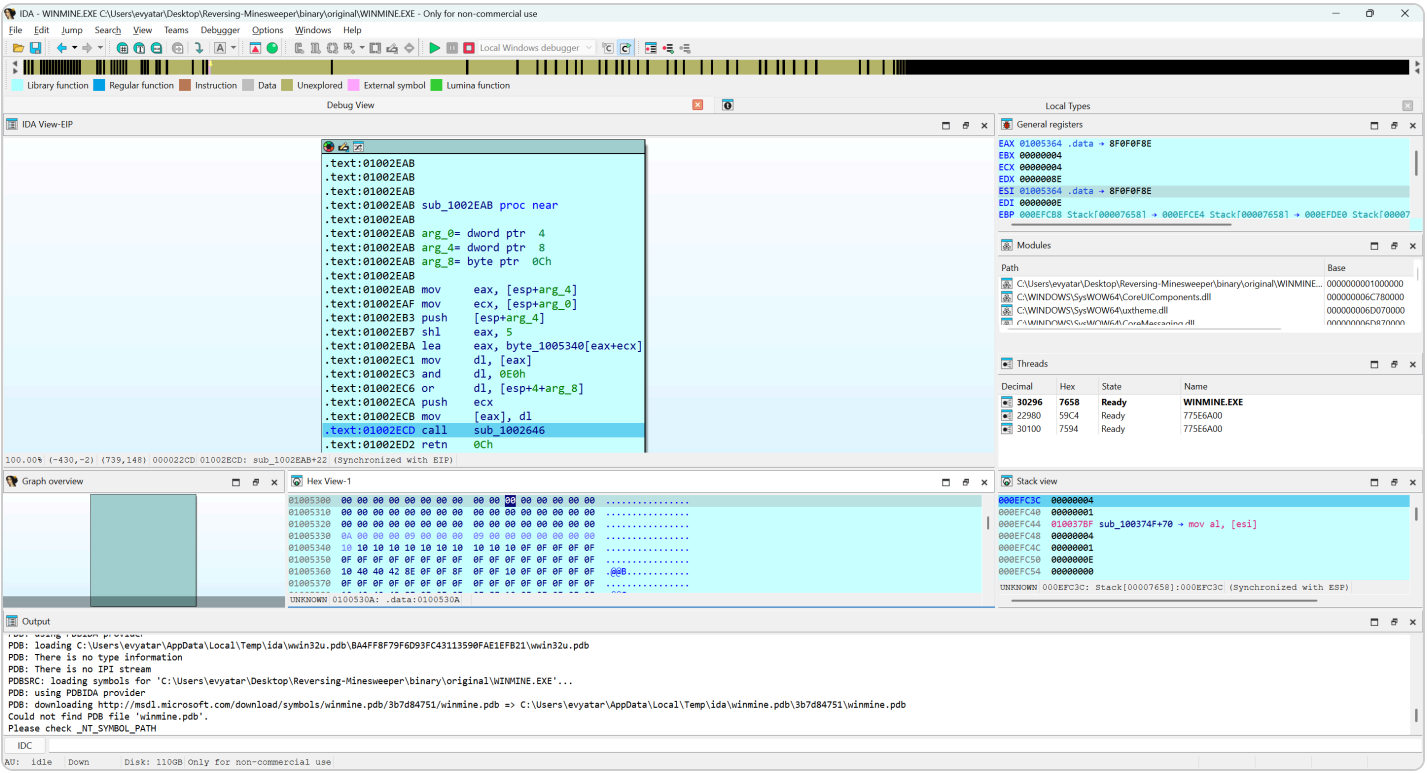
הזזה שמאלה ב - 5 ביטים היא כפל ב - 32, ולכן הבנתי שהמרחק בזיכרון בין שורה לשורה הוא `0x20` בתים. כלומר כתובת של תא מחושבת בצורה של:

```
כתובת הלוח + שורה * 32 + עמודה
```

כאשר כתובת הבסיס של הלוח היא: `0x01005340` והמרחק בין שתי שורות הוא: `0x20` חשוב לשים לב שהכתובת `0x01005340` היא תחילת כל המערך הפנימי של המשחק, כולל המסגרת שמקיפה את הלוח. התאים של המשחק עצמו מתחילים משורה 1 ועמודה 1, בגלל תאי הגבול שמסומנים ב - `0x10` . לכן התא הראשון שאפשר לשחק עליו נמצא ב:

```
0x01005340 + 1 * 32 + 1 = 0x01005361
```

כלומר הנוסחה מתייחסת לאינדקסים של המערך הפנימי, ולא לכך שהתא המשחקי הראשון יושב ממש בכתובת הבסיס. זה גם התאים למה שראיתי בפועל ב - Hex View כששיניתי תאים במשחק.



---

3. הערכים שתא בלוח יכול לקבל

אחרי שמצאתי את הלוח ניסיתי להבין איך המשחק שומר את המצב של כל תא. עשיתי את זה על ידי שינוי של תאים במשחק ובדיקה של הערכים שלהם בזיכרון. לדוגמה, תא רגיל ומכוסה היה: `0x0F` אחרי ששמתי עליו דגל הוא השתנה ל: `0x0E` ובתא שהיה בו מוקש ראיתי את המעבר:

```
0x8F -> 0x8E -> 0x8D -> 0x8F
```

כלומר: * `0x8F` - מוקש מכוסה * `0x8E` - מוקש עם דגל * `0x8D` - מוקש עם סימן שאלה אם משווים את `0x0F` ואת `0x8F` בבינארי:

```
0x0F = 0000 1111
0x8F = 1000 1111
```

רואים שההבדל היחיד הוא הביט `0x80`, ולכן הבנתי שהביט הזה קובע אם יש מוקש בתא.

המבנה של הבית

כל תא תופס בית אחד, כלומר 8 ביטים. בהתחלה חשבתי שכל ביט קובע משהו אחר, אבל כשהסתכלתי בקוד ראיתי שזה לא בדיוק ככה: שני הביטים העליונים הם דגלים, וחמשת הביטים הנמוכים הם שדה אחד שמחזיק ערך. נשאר באמצע ביט אחד, `0x20`, שלא הצלחתי לשייך לאף אחד מהשניים.

ביט	ערך	משמעות
7	`0x80`	יש מוקש בתא
6	`0x40`	התא פתוח (כבר לחצו עליו קליק שמאלי)
5	`0x20`	לא מצאתי לו שימוש
0–4	`0x1F`	שדה המצב של התא: מה מצויר בו, או כמה מוקשים יש סביבו כשהוא פתוח

כדי לבדוק את זה חיפשתי ב - IDA את המקומות בקוד שניגשים לכתובת של הלוח ועברתי עליהם. ראיתי הבדל שחוזר על עצמו: כשהמשחק שואל אם התא פתוח הוא בודק ביט בודד, וכשהוא צריך לדעת מה המצב של התא הוא לא בודק ביטים אלא לוקח את חמשת הביטים הנמוכים כמספר. בדיקת ביט בודד, למשל בכתובת `0x010033B4`:

```
test byte ptr [eax+ebx+1005340h], 40h
```

לעומת זה, בכתובת `0x0100301B` הקוד קורא את התא ועובד עם מסכה של חמישה ביטים:

```
movsx eax, byte ptr [esi+1005340h]
test  al, 40h                       ; האם התא פתוח - ביט בודד
and   eax, 1Fh                      ; חמשת הביטים הנמוכים כמספר
cmp   eax, 10h                      ; האם זה תא גבול
cmp   eax, 0Eh                      ; האם יש עליו דגל
```

אותה מסכה `and ..., 1Fh` חוזרת גם בכתובות `0x010033EA`, `0x010035CE` ו - `0x0100389B`. הכי ברור ראיתי את זה בקוד שמצייר את הלוח, בכתובת `0x01002669`. שם הערך אחרי המסכה משמש כאינדקס לתוך מערך של תמונות:

```
movsx edx, byte ptr [edx+eax+1005340h]
and   edx, 1Fh
push  dword ptr [edx*4+1005A20h]
```

בכתובת `0x01005A20` יש מערך של תמונות שהמשחק טוען כשהוא עולה. כלומר חמשת הביטים הנמוכים הם בעצם מספר התמונה שמצוירת בתא, ולכן זה שדה עם ערכים ולא ביטים נפרדים. זה גם מסביר למה תא לא יכול להיות גם

עם דגל וגם עם סימן שאלה - אלה שני ערכים של אותו שדה.

הביט `0x20`

הביט `0x20` הוא היחיד שלא הצלחתי לתת לו משמעות. הוא לא דגל, כי לא מצאתי בקוד שום מקום שבודק אותו לבד, משהו בסגנון `test ..., 20h` . הוא גם לא חלק משדה המצב, כי המסכה שהקוד משתמש בה היא `1Fh` , כלומר ביטים 0 עד 4 בלבד, והביט `0x20` נזרק בה החוצה. גם בבדיקות שעשיתי בזיכרון הוא אף פעם לא היה דלוק. הערך הגבוה ביותר שראיתי בשדה המצב עצמו, כלומר אחרי שמתעלמים מהביטים `0x80` ו `0x40` , הוא `0x10` של תא גבול. בכל שאר המצבים שעברתי עליהם הביט הזה כבוי, אז לפי מה שבדקתי הוא פשוט לא בשימוש בגרסה הזאת.

הערכים ההתחלתיים של הלוח

בפונקציה שמאתחלת את הלוח, זאת שנקראת מ - `0x010036B2` ממש לפני לולאת פיזור המוקשים, יש שתי כתיבות שקובעות את המצב ההתחלתי:

```
0x01002EDB  mov byte ptr [eax+1005340h], 0Fh    ; תא משחק רגיל, מכוסה
0x01002F04  mov byte ptr [eax+1005340h], 10h    ; תא גבול מסביב ללוח
```

הראשונה ממלאת את כל שטח המשחק ב - `0x0F` , והשנייה שמה `0x10` בשורות ובעמודות שמסביב. תאי ה - `0x10` הם הגבול של הלוח ולא תאים שאפשר לשחק בהם. אני מניח שהם קיימים כדי שהקוד שסופר מוקשים שכנים יוכל לגשת לשכנים של תאי הקצה בלי לבדוק גבולות כל פעם.

חמשת הביטים הנמוכים

חמשת הביטים הנמוכים משמשים ביחד כדי לתאר את מצב התא.

ערך נמוך	משמעות
`0x0` – `0x8`	בתא פתוח - מספר המוקשים שנמצאים לידו
`0x9`	מצב זמני בזמן שלוחצים על תא שמסומן בסימן שאלה
`0xA`	מוקש שנחשף בסיום משחק לאחר הפסד
`0xB`	דגל שהונח על תא שאין בו מוקש
`0xC`	המוקש שעליו לחצו והוא התפוצץ
`0xD`	סימן שאלה
`0xE`	דגל

ערך נמוך	משמעות
`0xF`	תא מכוסה
`0x10`	תא גבול, מחוץ לשטח המשחק

יש גם מצבים זמניים בזמן הלחיצה. כשמחזיקים לחיצה על תא מכוסה, הערך הנמוך שלו משתנה מ - `0xF` ל - `0x0` , וכשמשחררים בלי לפתוח אותו הוא חוזר ל - `0xF` . בצורה דומה, תא עם סימן שאלה עובר בזמן הלחיצה מ - `0xD` ל - `0x9` , וכשמשחררים הוא חוזר ל - `0xD` . אחר כך בדקתי גם הפסד, דגל שגוי ולחיצה ממושכת, וככה מצאתי את שאר הערכים שמופיעים כאן. הערך הסופי של התא מתקבל משילוב של הביטים האלה. לדוגמה:

```
0x0F = תא רגיל מכוסה
0x8F = מוקש מכוסה

0x0E = תא רגיל עם דגל
0x8E = מוקש עם דגל

0x0D = תא רגיל עם סימן שאלה
0x8D = מוקש עם סימן שאלה
```

בתא פתוח הביט `0x40` דולק, והערך הנמוך מציין כמה מוקשים נמצאים ליד התא:

```
0x40 = תא פתוח עם 0 מוקשים לידו
0x41 = תא פתוח עם מוקש אחד לידו
0x42 = תא פתוח עם 2 מוקשים לידו
0x43 = תא פתוח עם 3 מוקשים לידו
...
0x48 = תא פתוח עם 8 מוקשים לידו
```

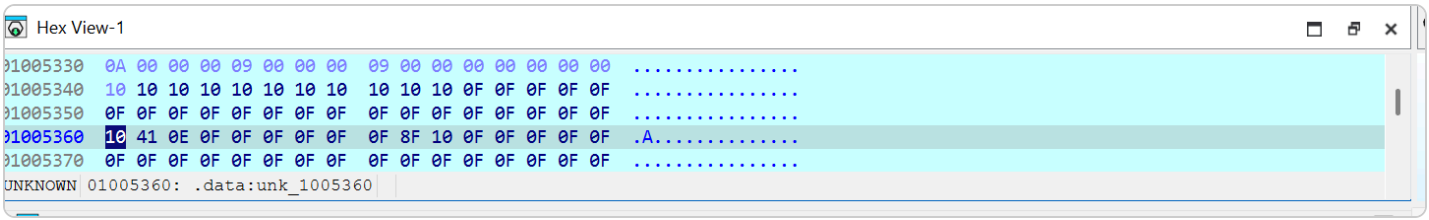
בבדיקה שלי ראיתי בפועל תאים בערכים `0x40` עד `0x43` , אבל לפי הקוד אותו שדה יכול להגיע עד `0x48` . יש עוד כמה ערכים שמופיעים בסיום משחק. למשל:

```
0x8A = מוקש שנחשף אחרי הפסד
0x0B = דגל שהונח על תא שאין בו מוקש
0xCC = המוקש שעליו לחצו והוא התפוצץ
```

בסופו של דבר, בשביל הפאצ' של השלב הזה הדבר החשוב ביותר היה להבין את ההבדל בין:

```
0x8F = מוקש מכוסה
0x8E = מוקש עם דגל
```

ברגע שמצאתי את ההבדל הזה היה אפשר לשנות את הקוד שמניח את המוקשים ולגרום לו ליצור אותם ישר כ - `0x8E` . ככה כל המוקשים מופיעים עם דגל כבר בתחילת המשחק.



שלב ב' - ניצחון מהיר

בשלב הזה המטרה הייתה לגרום למשחק להתחיל עם מוקש אחד בלבד, כשהוא כבר מסומן בדגל. לא שיניתי את השעון, אלא רק את מספר המוקשים.

4. איפה נקבע גודל לוח המשחק

לגודל הלוח הגעתי דרך הלולאה מסעיף 1, בלי לחפש מאפס. הלולאה שמפזרת את המוקשים קוראת את הרוחב והגובה משני משתנים:

```
0x010036C7  push dword ptr [1005334h]     ; רוחב
0x010036D2  push dword ptr [1005338h]     ; גובה
```

משם המשכתי אחורה עם ה - xrefs של IDA ובדקתי מי כותב לשני המשתנים האלה. מצאתי שזה קורה ממש לפני הלולאה, באותה פונקציה:

```
0x0100367A  mov eax, [010056ACh]
0x0100367F  mov ecx, [010056A8h]
0x010036A7  mov [01005334h], eax
0x010036AC  mov [01005338h], ecx
```

כלומר בכל משחק חדש המשחק מעתיק את הגודל משני משתנים אחרים:

```
0x010056A8 - גובה הלוח
0x010056AC - רוחב הלוח
```

באלה נשמר הגודל שנבחר לפי רמת הקושי. עשיתי xref גם עליהם, וזה הוביל אותי לקוד שקורא מטבלה שמתחילה ב - `0x01005010` , ובה הנתונים של שלוש רמות הקושי:

רמה	מוקשים	גובה	רוחב
Beginner	10	9	9
Intermediate	40	16	16
Expert	99	16	30

הקוד שקורא מהטבלה נמצא בכתובת `0x01001E70`, ואפשר לראות בו שהמשחק לוקח ממנה שלושה ערכים ושומר אותם במשתנים המתאימים:

```
mov ecx, [eax+01005010h]
mov [010056A4h], ecx

mov ecx, [eax+01005014h]
mov eax, [eax+01005018h]

mov [010056A8h], ecx
mov [010056ACh], eax
```

כלומר בכל שורה בטבלה יש מספר מוקשים, גובה ורוחב. כשמתחיל משחק חדש, הרוחב והגובה נטענים שוב:

```
0x0100367A    mov eax, [010056ACh]
0x0100367F    mov ecx, [010056A8h]
```

ובהמשך הם נשמרים ב:

```
0x01005334 - רוחב הלוח
0x01005338 - גובה הלוח
```

בשלב הזה לא הייתי צריך לשנות את גודל הלוח, רק להבין איפה הוא נקבע.

---

5. איפה נקבע מספר המוקשים

מספר המוקשים של רמת הקושי הנוכחית נשמר בכתובת: `0x010056A4` לדוגמה, ב - Beginner הערך הוא 10, ב - Intermediate הוא 40 וב - Expert הוא 99. בתחילת משחק חדש מצאתי את ההוראה הבאה בכתובת `0x010036B7`:

```
mov eax, [010056A4h]
```

היא טוענת ל - `EAX` את מספר המוקשים. מיד אחר כך הערך נשמר ב - : `0x01005330` בהמשך יש לולאה שמניחה מוקשים על הלוח. אחרי כל מוקש שהמשחק מניח הוא מבצע:

```
dec dword ptr [01005330h]
```

והלולאה ממשיכה כל עוד הערך לא הגיע לאפס. מכאן היה ברור שאם במקום לטעון את מספר המוקשים הרגיל אכניס את הערך `1` , הלולאה תיצור מוקש אחד בלבד.

---

6. הפאצ' למוקש יחיד

בכתובת: `0x010036B7` ההוראה המקורית הייתה:

```
mov eax, [010056A4h]
```

כלומר טעינה של מספר המוקשים לפי רמת הקושי. שיניתי אותה ל:

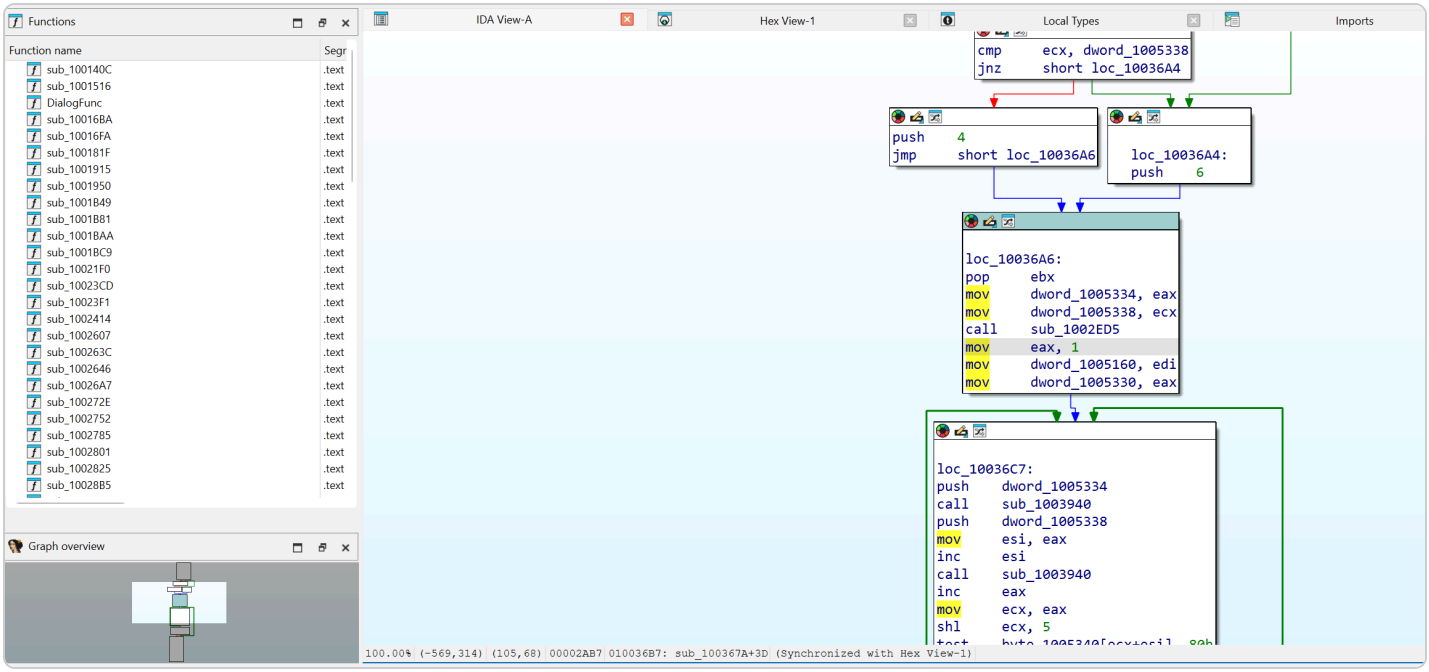
```
mov eax, 1
```

הבתים השתנו מ:

```
A1 A4 56 00 01
```

ל:

```
B8 01 00 00 00
```

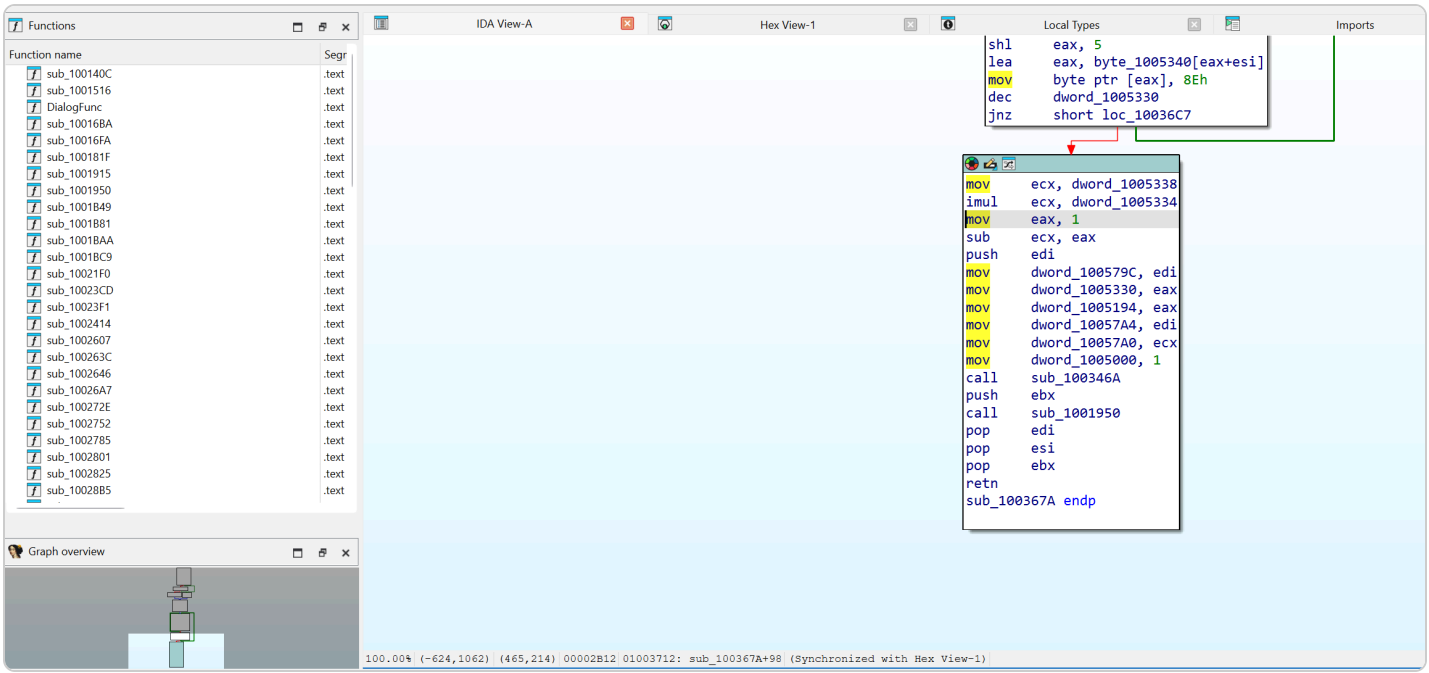


בהמשך הפונקציה מספר המוקשים נטען פעם נוספת, בכתובת: `0x01003712` גם שם הייתה אותה הוראה:

```
mov eax, [010056A4h]
```

ושיניתי גם אותה ל:

```
mov eax, 1
```



שני המקומות יושבים בקובץ ב - offset `0x2AB7` ו - `0x2B12` . כאן הבתים המקוריים `A1 A4 56 00 01` מופיעים בקובץ בדיוק פעמיים, ואלה בדיוק שתי הטעינות האלה. הייתי צריך לשנות גם את המקום השני כי הערך שם לא

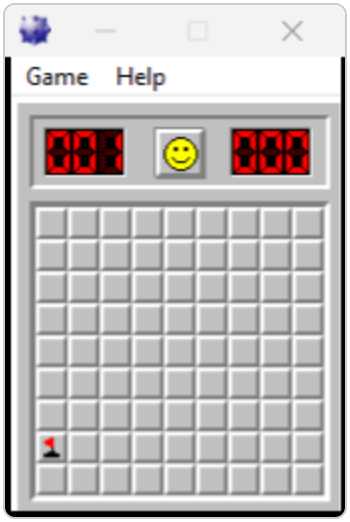
משמש לפיזור המוקשים אלא לחשבונאות של המשחק. אפשר לראות את זה בקוד שמיד לפניו ומיד אחריו:

```
0x01003705  mov  ecx, [1005338h]     ; גובה הלוח
0x0100370B  imul ecx, [1005334h]     ; כל התאים בלוח = רוחב * גובה
0x01003712  mov  eax, [10056A4h]     ; מספר המוקשים
0x01003717  sub  ecx, eax            ; מספר התאים הבטוחים
0x01003720  mov  [1005330h], eax     ; מונה המוקשים שמוצג על המסך
0x01003730  mov  [10057A0h], ecx     ; כמה תאים צריך לפתוח כדי לנצח
```

כלומר הערך הזה קובע גם את המספר שמופיע במונה המוקשים וגם כמה תאים צריך לפתוח כדי לנצח. אם הייתי משנה רק את הטעינה הראשונה, המשחק היה מפזר מוקש אחד אבל היה ממשיך להתנהג כאילו יש עשרה, והמונה היה מראה 010. לכן שיניתי גם את המקום השני. בנוסף השארתי את הפאצ' שעשיתי בשלב א':

```
0x010036FA    mov byte ptr [eax], 8Eh
```

לכן המוקש היחיד שנוצר גם מסומן ישר בדגל. אחרי השינוי הרצתי את המשחק וקיבלתי לוח עם מוקש אחד בלבד, כשהמונה מראה `001` והמוקש כבר מסומן.



שיחקתי את זה עד הסוף. פתחתי תא אחד רחוק מהמוקש, וכיוון שאין מוקשים אחרים כל הלוח נפתח בבת אחת והגעתי לניצחון אחרי שנייה אחת:



הדגל על המוקש היחיד נשאר במקומו, ובשעון לא נגעתי.

שלב ג' - שהעולם יידע מי אתם

בשלב הזה המטרה הייתה להוסיף `MessageBox` שיופיע לפני שהמשחק מתחיל. אחרי שלוחצים על OK המשחק צריך להמשיך לרוץ כרגיל, כולל הפאצ' שעשיתי בשלב א'.

7. הוספת MessageBox

קודם בדקתי מה ה - Entry Point המקורי של התוכנית, כלומר מאיפה הקוד מתחיל לרוץ כשפותחים את ה - EXE. מצאתי שה - Entry Point המקורי הוא: `0x01003E21` וה - RVA שלו הוא: `0x3E21` לא רציתי לדרוס את הקוד המקורי, לכן חיפשתי מקום פנוי בקובץ שבו אפשר להכניס את הקוד החדש. מצאתי מקום פנוי בכתובת: `0x01004A58` שה - RVA שלו הוא: `0x4A58` לאחר מכן שיניתי את `AddressOfEntryPoint` מ:

```
0x3E21
```

ל:

```
0x4A58
```

כך שכאשר מפעילים את המשחק, התוכנית מתחילה קודם מהקוד שהוספתי. בקובץ כבר היה import של `MessageBoxW`, ולכן לא הייתי צריך להוסיף import חדש. הכתובת שלו בIAT היא: `0x010010B8` הקוד שהוספתי הוא:

```
push 0
push offset L"Cyber"
push offset L"Hacked by Evyatar"
```

```
push 0
call dword ptr [0x010010B8]
jmp 0x01003E21
```

בחרתי שהכותרת תהיה: `Cyber` והטקסט בהודעה יהיה: `Hacked by Evyatar` ארבעת ה - `push` מעבירים את הפרמטרים ל - `MessageBoxW` . אחרי שההודעה מוצגת, הקוד מבצע:

```
jmp 0x01003E21
```

כלומר חוזר ל - Entry Point המקורי של המשחק וממשיך את ההרצה הרגילה.

התאמת ה - section לקוד החדש

הקוד החדש נמצא בתוך אזור `.text` , אבל המקום שבו הכנסתי אותו היה מעבר ל - `VirtualSize` המקורי של ה - section. ה - VirtualSize המקורי היה:

```
0x3A56
```

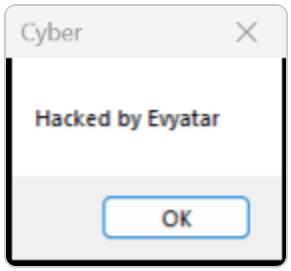
ושיניתי אותו ל:

```
0x3C00
```

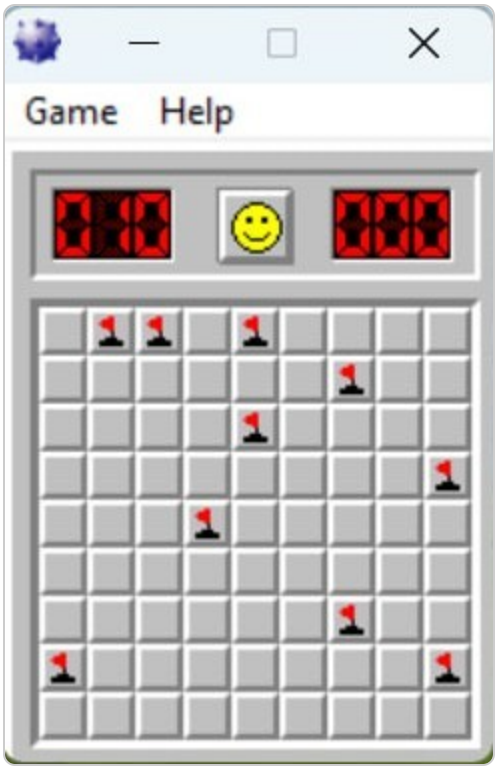
כך שהאזור שבו נמצא הקוד החדש יהיה ממופה לזיכרון כשהתוכנית עולה. בסך הכול השינויים העיקריים היו:

מה שיניתי	לפני	אחרי
Entry Point	`0x3E21`	`0x4A58`
`.text VirtualSize`	`0x3A56`	`0x3C00`

את השינויים ב - Entry Point וב - VirtualSize ביצעתי דרך CFF Explorer. אחרי השינוי הרצתי את הקובץ וה - MessageBox הופיע לפני פתיחת המשחק:



אחרי שלחצתי על OK המשחק המשיך לרוץ כרגיל. השארתי בקובץ גם את הפאצ' של שלב א', ולכן אחרי סגירת ה - MessageBox המשחק עולה כשהמוקשים כבר מסומנים בדגל.



כך ה - MessageBox רץ קודם, ולאחר מכן המשחק ממשיך לקוד המקורי בלי לפגוע בפעולה שלו.

שלב ד' - בתחבולות תעשה לך מלחמה

בשלב הזה המטרה הייתה להוסיף ל - EXE קוד שפונה לשרת HTTP מקומי שרץ על `127.0.0.1:80` ושולח אליו מסר.

8. פנייה לשרת HTTP

קודם הרמתי שרת HTTP פשוט ב - Python שמאזין על: `127.0.0.1:80` השרת שומר כל בקשה שמגיעה אליו ב - log, וככה יכולתי לבדוק אם הבקשה באמת נשלחת מתוך המשחק.

איך עשיתי את הפנייה

הבעיה הראשונה הייתה שב - WINMINE אין import של פונקציות WinINet שאפשר להשתמש בהן ישירות לבקשת HTTP. כן מצאתי שבקובץ כבר קיימות שתי פונקציות שיכולות לעזור:

```
LoadLibraryA
GetProcAddress
```

הכתובות שלהן ב - IAT הן:

```
LoadLibraryA   = 0x0100109C
GetProcAddress = 0x01001094
```

במקום לערוך את טבלת ה - imports ולהוסיף אליה את WinINet, החלטתי לטעון את `wininet.dll` בזמן ריצה ולמצוא את הפונקציות שאני צריך בעזרת `GetProcAddress` . הקוד החדש מתחיל בכתובת: `0x01004A58` בהתחלה אני טוען את `wininet.dll` :

```
push 01004AD0h
call dword ptr [0100109Ch]
mov esi, eax
```

בכתובת `0x01004AD0` שמורה המחרוזת:

```
wininet.dll
```

`LoadLibraryA` מחזירה handle ל - DLL ב - EAX, ושמרתי אותו ב - ESI כדי להשתמש בו בקריאות הבאות.
אחר כך מצאתי את הכתובת של `InternetOpenA` :

```
push 01004ADCh
push esi
call dword ptr [01001094h]
```

המחרוזת `InternetOpenA` נמצאת ב: `0x01004ADC` אחרי `GetProcAddress` , הכתובת של הפונקציה נמצאת ב - EAX. לאחר מכן קראתי ל - `InternetOpenA` :

```
push 0
push 0
push 0
push 1
push 0
call eax
mov edi, eax
```

ב - x86 הפרמטרים נדחפים למחסנית בסדר הפוך. כאן השתמשתי ב - access type עם הערך `1` , ושאר הפרמטרים שלא הייתי צריך קיבלו `0` . ה - handle שחזר מ־`InternetOpenA` נשמר ב - EDI. אחר כך השתמשתי שוב ב - `GetProcAddress` , הפעם כדי למצוא את `InternetOpenUrlA` :

```
push 01004AEAh
push esi
call dword ptr [01001094h]
```

המחרוזת `InternetOpenUrlA` נמצאת ב - `0x01004AEA` : ה - URL ששמרתי בתוך הקובץ נמצא בכתובת: `0x01004AFB` והוא:

```
http://127.0.0.1/pwned_by_Evyatar
```

לאחר מכן קראתי ל - `InternetOpenUrlA` :

```
push 0
push 0
push 0
push 0
push 01004AFBh
push edi
call eax
```

גם כאן דחפתי את הפרמטרים בסדר הפוך. ה - handle שקיבלתי מ - `InternetOpenA` הוא הפרמטר הראשון של הפונקציה, וה - URL הוא הפרמטר השני. הנתיב:

```
/pwned_by_Evyatar
```

הוא המסר שבחרתי לשלוח לשרת. כל החלק של בקשת ה - HTTP בקובץ נראה כך:

```
01004A58  push 01004AD0h
01004A5D  call dword ptr [0100109Ch]
01004A63  mov esi, eax

01004A65  push 01004ADCh
01004A6A  push esi
01004A6B  call dword ptr [01001094h]

01004A71  push 0
01004A73  push 0
01004A75  push 0
01004A77  push 1
01004A79  push 0
01004A7B  call eax
01004A7D  mov edi, eax
```

```
01004A7F  push 01004AEAh
01004A84  push esi
01004A85  call dword ptr [01001094h]

01004A8B  push 0
01004A8D  push 0
01004A8F  push 0
01004A91  push 0
01004A93  push 01004AFBh
01004A98  push edi
01004A99  call eax
```

המשך ההרצה של המשחק

אחרי בקשת ה - HTTP הקוד ממשיך ישירות לקוד של ה - MessageBox משלב ג׳:

```
01004A9B  push 0
01004A9D  push 01004B1Dh
01004AA2  push 01004B29h
01004AA7  push 0
01004AA9  call dword ptr [010010B8h]
```

`0x010010B8` היא כתובת ה - IAT של `MessageBoxW` . אחרי שסוגרים את ההודעה, הקוד חוזר ל - Entry Point המקורי של המשחק:

```
01004AAF  jmp 01003E21h
```

כלומר הסדר בפועל הוא:

```
טעינת wininet.dll
        ↓
מציאת InternetOpenA
        ↓
פתיחת WinINet session
        ↓
מציאת InternetOpenUrlA
        ↓
שליחת הבקשה לשרת המקומי
        ↓
MessageBox משלב ג'
```

```
        ↓
חזרה ל - Entry Point המקורי
```

הקשיים שהיו לי

הקושי העיקרי היה שפונקציות WinINet לא נמצאות בטבלת ה - imports המקורית של המשחק. במקום להוסיף import חדש ידנית, השתמשתי ב - `LoadLibraryA` וב - `GetProcAddress` שכבר היו קיימות בקובץ, וטענתי את הפונקציות בזמן ריצה. קושי נוסף היה להעביר את הפרמטרים נכון. הקובץ הוא x86, ולכן בכל קריאה הייתי צריך לדחוף את הפרמטרים למחסנית בסדר הפוך. במיוחד ב - `InternetOpenUrlA` , שמקבלת שישה פרמטרים. בנוסף הייתי צריך למצוא מקום גם לקוד החדש וגם למחרוזות:

```
wininet.dll
InternetOpenA
InternetOpenUrlA
http://127.0.0.1/pwned_by_Evyatar
```

ובסוף היה חשוב לחזור לזרימה המקורית של המשחק כדי שהוספת הקוד לא תמנע ממנו להיפתח ולעבוד.

בדיקת התוצאה

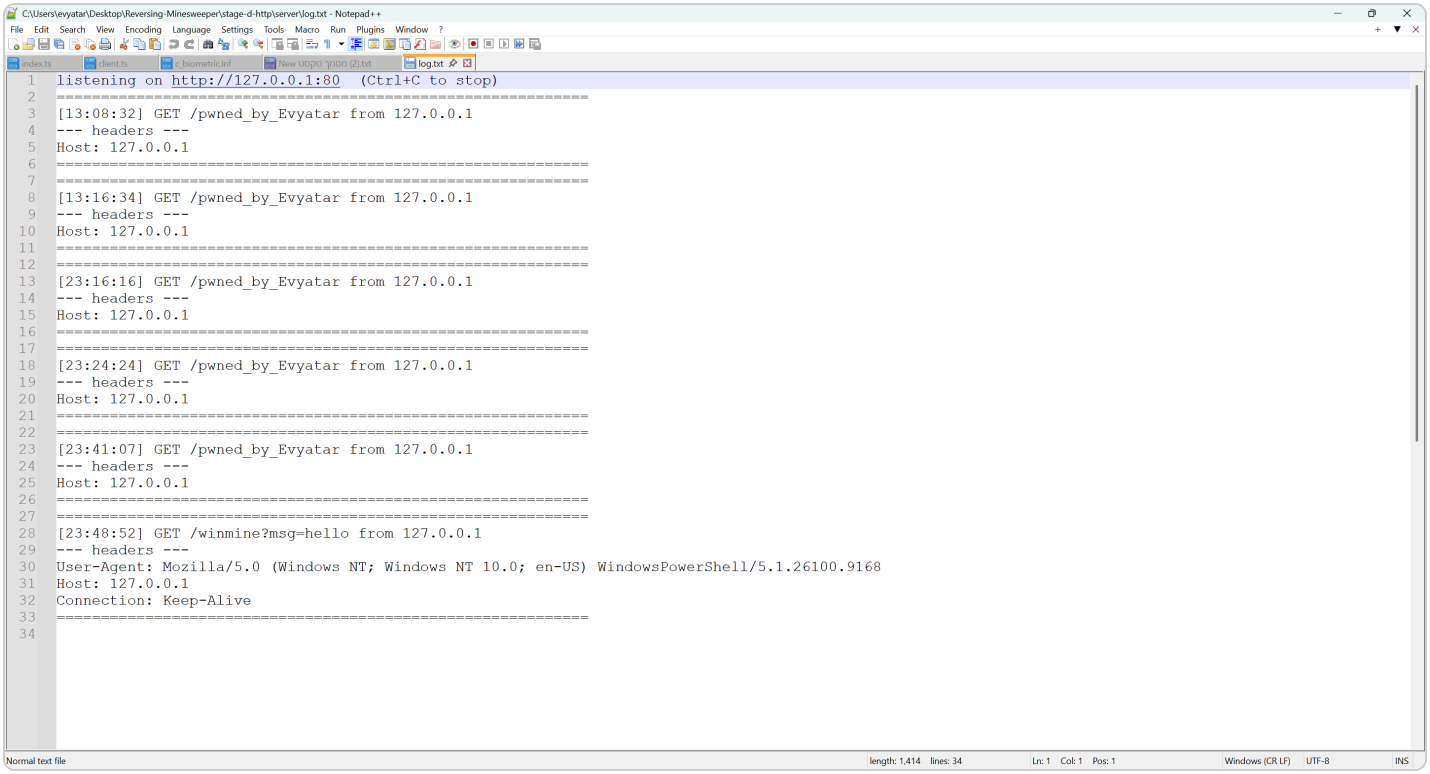
אחרי שהפעלתי את השרת הרצתי את הקובץ של שלב ד׳. ב - log התקבלו בקשות כמו:

```
GET /pwned_by_Evyatar from 127.0.0.1
```

וגם:

```
Host: 127.0.0.1
```

כלומר הבקשה באמת הגיעה מתוך המחשב לשרת המקומי על פורט 80.



(השורה `GET /winmine?msg=hello` שמופיעה בסוף ה - log היא בדיקה ידנית שעשיתי לשרת לפני כן ולא נשלחה מתוך ה - EXE. הבקשות של הפתרון הן אלה שפונות ל - `/pwned_by_Evyatar`.) אחרי שליחת הבקשה גם ה - MessageBox משלב ג׳ הופיע והמשחק המשיך לרוץ, כך שהקוד שהוספתי לא פגע בהמשך הפעולה של המשחק.